

Artificial Neural Networks. Assignment 2.

Link to the [GitHub](#)

Link to the [GoogleColab](#)

Taly Shasho Frydman 345317218

Aleksandra Oistrakh 342743689

1. CNN debugging

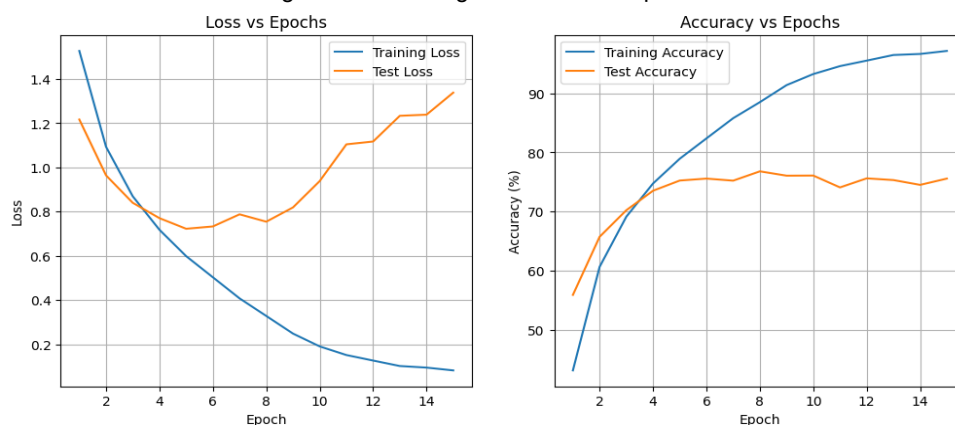
Model evaluation: Figure 1.1: Classes received by code



Figure 1.2: Training loss training accuracy and test accuracy:

Metric	Beginning	Best / Final Value	Interpretation
Training Loss	~1.5	~0.1	Consistently decreased, showing the model learned the training data very well.
Training Accuracy	~43%	~97%	Continuously increased throughout training.
Test Accuracy	~56%	~75% (Epoch 7)	Improved rapidly at first, then stopped improving.
Test Loss	Decreased initially	Increased after Epoch 7	Indicates worsening performance on unseen data.

Figure 1.3 Learning curves across epochs:

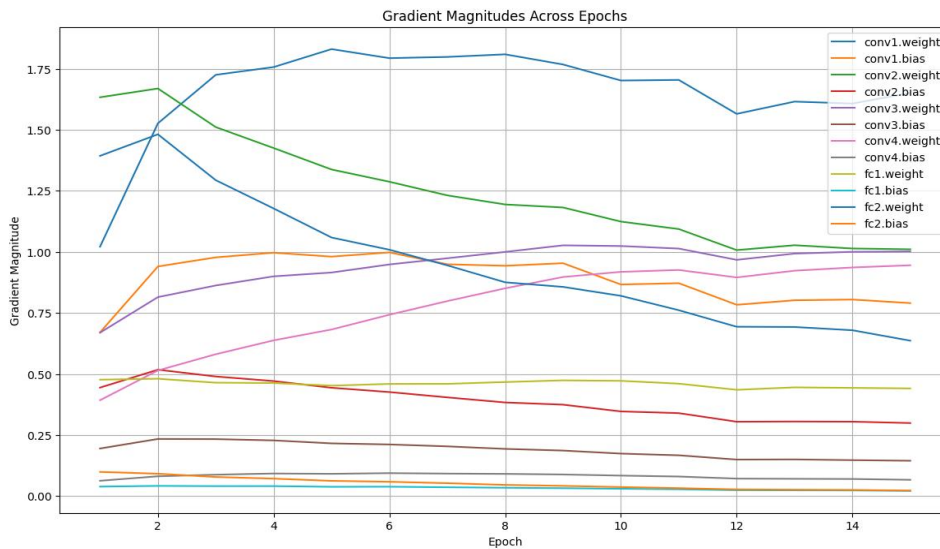


Gradient magnitudes of each layer within each epoch during training:

The gradient magnitude plots demonstrate the vanishing gradient problem inherent to the initial architecture. Due to the selection of the Sigmoid activation function, which saturates quickly at its extremes, the gradient magnitudes for the early convolutional layers (such as conv1.weight and conv1.bias) compress significantly and remain critically close to zero throughout all 15 epochs. As a consequence, these initial layers are unable to effectively update their weights and capture

foundational low-level features (like edges and shapes). This gradient bottleneck directly explains the model's limited test accuracy and the severe overfitting observed in the learning curves. This visual behavior reveals that the initial network suffers heavily from the vanishing gradient problem. Since the early layers are blocked by saturating Sigmoid activations, backpropagation fails to deliver meaningful updates to the beginning of the network. True optimization stability and healthy backpropagation are only achieved later, once these architectural bottlenecks are resolved by transitioning to ReLU activations.

Figure 1.4. Gradient Magnitudes of the Initial Model:



Model improvement:

Problems:

1. After around epoch 8, the model started overfitting: it kept improving on training data but not on test data.
2. Classes missed: the displayed examples come from a random batch, so we get duplicate classes and miss ones, also the code can be misinterpreting those classes for different ones.
3. Misclassified when flipped horizontally to Horses and frogs were.

Fix:

1. After the first convolutional layer a Batch Normalization was added, this stabilizes the activations during training by keeping their distribution more consistent between batches. This helps the network learn more efficiently, given that it stabilises training and improves gradient flow, which helps to slightly reduce overfitting.
2. Changed the visualization function to intentionally search the test set and display one correctly classified example from each CIFAR-10 class. This guarantees that all 10 categories appear. Moreover, a per-class accuracy evaluation was added to confirm that all classes are performing well
3. Added random horizontal flips, this way the CNN now trains on both normal and flipped images, which helps it learn orientation invariant features.

Figure 1.5: Classes after code fix:



After these changes the model was still overfitting. This was clear because training accuracy hit 92% while test accuracy flatlined at 79%, with the model failing on complex classes like cats (58.5% accuracy). The gradient graph confirmed this: the final decision-making layers (fc1/fc2) had flatlined near zero, leaving the first layer to do all the heavy lifting and memorization.

To fix this, we added Dropout layers. By randomly shutting off neural pathways during training, dropout prevents the network from relying on a single shortcut. It forces the lazy final layers to wake up, distribute the learning load, and focus on extracting robust, generalized features, closing the gap between training and testing performance, getting 63.2% accuracy on cat class, the lowest one and 80.3% on general test accuracy.

Fig 1.6. Improved learning curves across epochs:

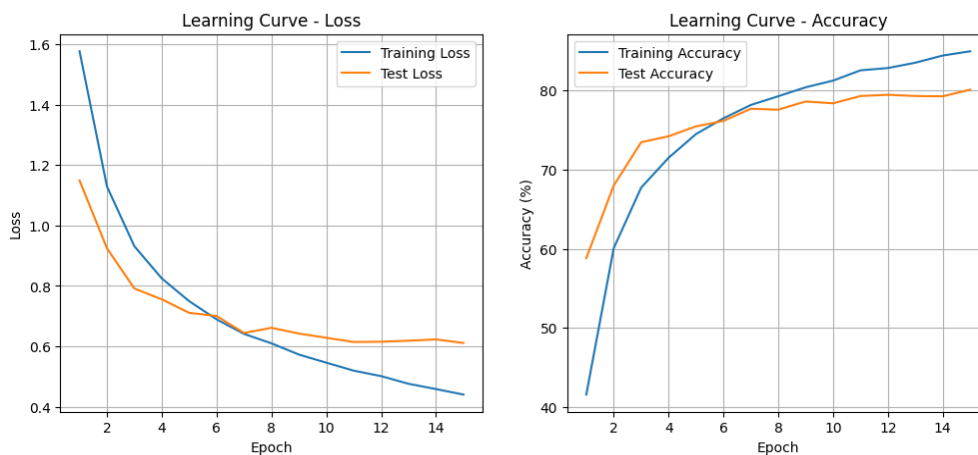
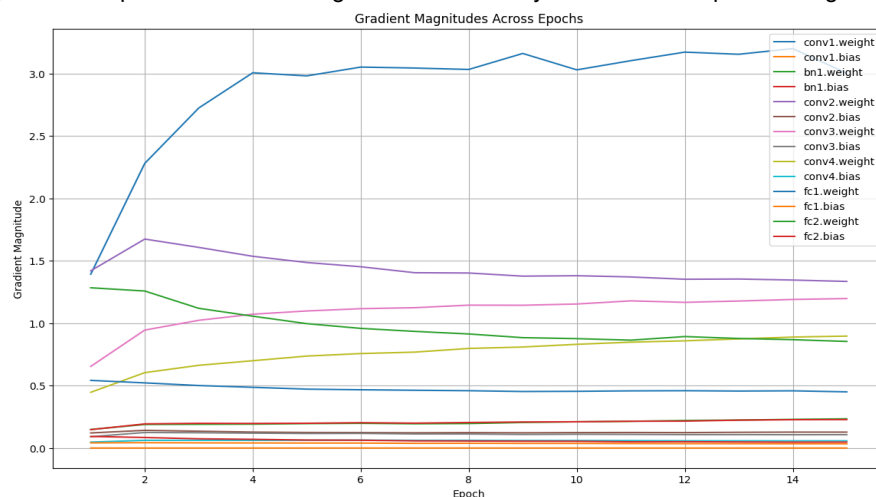


Figure 1.7. Improved Gradient magnitudes of each layer within each epoch during training:



Comparison of results before and after fixing the code

The improved model performed better than the original CNN, especially in terms of generalization and stability during testing. In the original model as explained before we saw signs of severe overfitting. In contrast, the improved model showed much healthier behavior. The training accuracy increased more gradually and reached about 85%, while the test accuracy improved steadily to approximately 80.35%. More importantly, the training and test curves remained much closer together throughout training, indicating that the model generalized substantially better and overfitted much less.

The loss curves show this improvement. In the original model, the test loss initially decreased but then rose dramatically from about 0.7 to over 1.3 in later epochs, meaning the model became increasingly confident on the training set while performing worse on unseen data. In the improved model, both the training and test loss curves decreased more smoothly, and the test loss remained relatively stable near 0.6 instead of exploding upward. This indicates that the added batch normalization layer stabilized the learning process and prevented the network from memorizing noisy training patterns. The horizontal flip augmentation also contributed by exposing the model to different object orientations during training, reducing sensitivity to mirrored images.

The accuracy curves also demonstrate the difference between the two architectures. In the original model, the training accuracy continued climbing aggressively while the test accuracy plateaued early, producing a very large generalization gap of more than 20 percentage points. In the improved model, the gap between training and test accuracy became much smaller, showing that the network learned more transferable visual features rather than simply memorizing examples. Although the final training accuracy of the improved model was lower, its higher test accuracy proves that it is a better model overall because performance on unseen data is the true important measure.

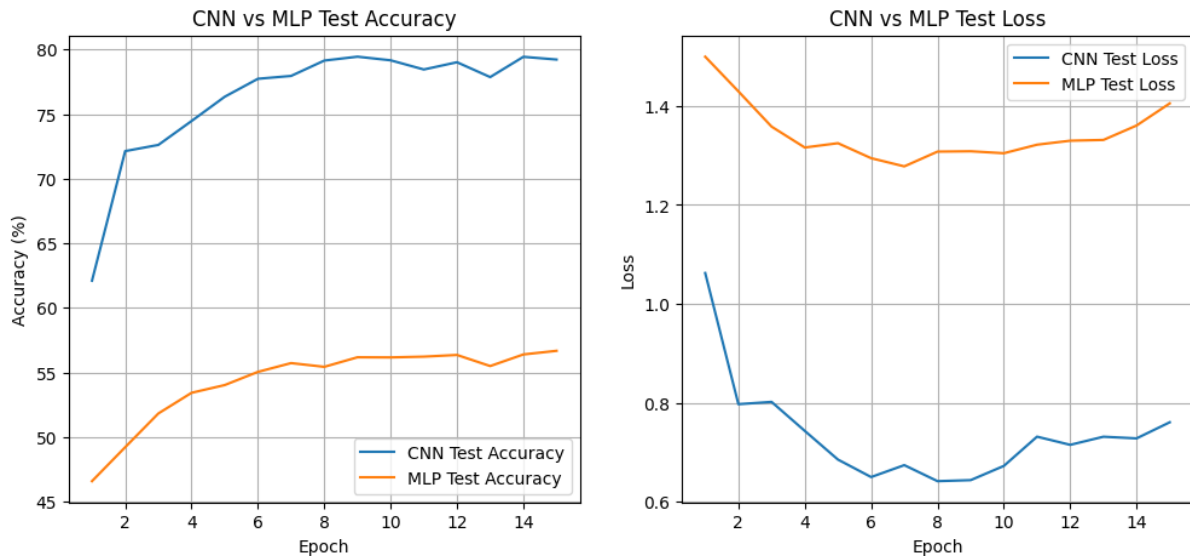
The gradient magnitude graphs also reveal important differences between the two models. In the original network, several gradient curves fluctuated strongly and decreased irregularly across epochs, especially in the deeper layers. This unstable behavior reflects noisier optimization and stronger overfitting. In the improved model, the gradients became smoother and more controlled across training. While some gradients became smaller than before, this does not indicate failed learning; rather, it suggests that the network converged more smoothly and required less aggressive corrections during optimization.

The per class accuracies further support the improvement. In the original model, certain classes such as airplanes and birds were missing from predictions, while flipped horses and frogs caused consistent classification errors. After the improvements, the model no longer showed problems with flipped images, indicating that the horizontal flip augmentation successfully reduced orientation sensitivity. In addition, all classes are now correctly represented and recognized more consistently across predictions. Overall, the improved architecture achieved better generalization, reduced overfitting, more stable gradients, and higher test accuracy.

2. Comparison to MLP

	<i>Final Test Accuracy</i>	<i>Parameters</i>	<i>Model Memory</i>	<i>Training Time</i>
<i>CNN</i>	78.67%	653450	2.49 MB	294.18 sec
<i>MLP</i>	56.19%	1738890	6.63 MB	249.27 sec

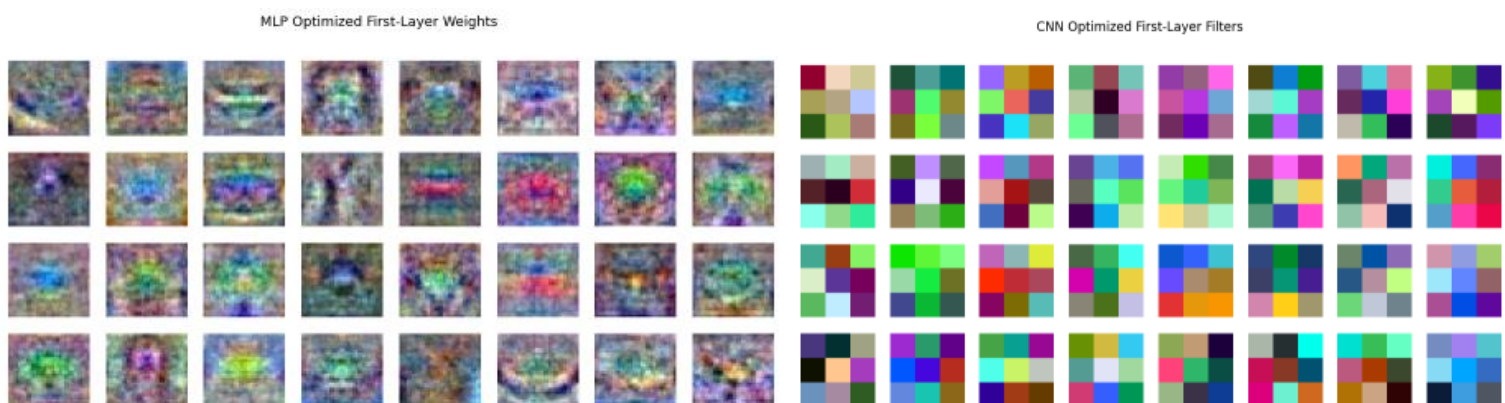
Figure 2.1. Test accuracy and loss comparison



The CNN achieved higher test accuracy because CNNs preserve spatial information and learn meaningful visual features such as edges, textures, and shapes, whereas the MLP flattens the image and loses spatial structure. The CNN also achieved lower test loss and generalized much better to unseen images.

Despite performing worse, the MLP used far more parameters and memory compared to CNN. MLP trained faster because fully connected operations are computationally simpler than convolutions. The weight visualizations also showed that CNN filters learned structured and interpretable visual patterns, while the MLP weights appeared noisy and less meaningful. Overall, the CNN provided better accuracy, better generalization, more interpretable features, and more efficient parameter usage, making it the superior architecture for image classification.

Figure 2.1. First layers of CNN and MLP.



The visualization of optimized weights highlights major interpretability differences between the architectures. In the CNN visualization, the first layer filters appear as structured color and edge detectors. Many filters resemble simple visual patterns such as color transitions or directional edges. This indicates that CNN learned meaningful low level visual features directly related to image structure. In contrast, the MLP weight visualizations appear noisy and blurry with far less recognizable structure. Since the MLP processes flattened vectors instead of spatial image regions, its learned weights are more difficult to interpret visually and do not correspond clearly to meaningful image features.

MLPs are traditional, fully connected networks, they are good at handling tabular data, such as spreadsheets or fixed feature vectors, where individual data is independent of their physical position given that MLP flattens the inputs, it loses track of spatial location and suffer from an explosion of parameters when dealing with large inputs like images. CNNs solve this problem by using small, sliding filters that scan inputs locally, making them the standard for image classification, and audio processing. This design allows CNNs to preserve spatial relationships and achieve spatial invariance, meaning they can efficiently recognize shapes, edges, and textures no matter where they appear in the visual frame. They are used for object detection, facial recognition, and medical imaging.

3. Overfitting Analysis (large VS small model)

<i>Model</i>	<i>Model capacity</i>	<i>Convolutional layers</i>	<i>Filters</i>	<i>Fully connected layer</i>
<i>Small CNN</i>	Lower	2	32, 64	128 neurons
<i>Large CNN</i>	Higher	4	64, 128, 256, 512	512 neurons

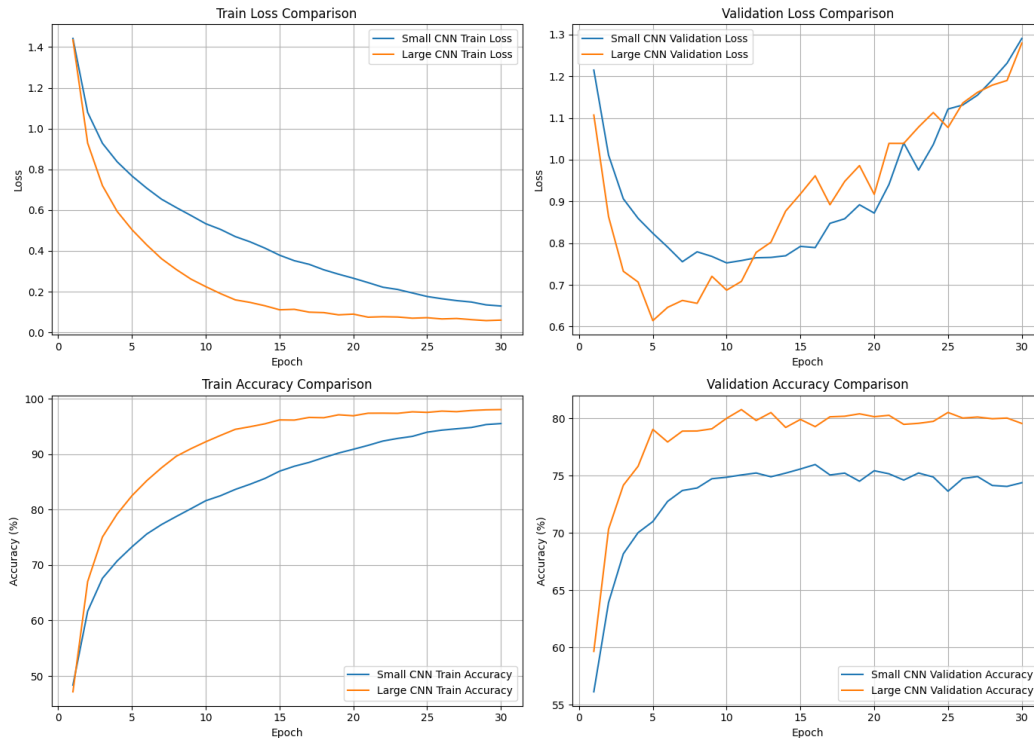
Both models were trained for 30 epochs under the same training conditions in order to make a fair comparison.

The results show that the Large CNN learned the training data much more aggressively. Its training accuracy reached 97.97%, compared to 95.75% for the Small CNN, and its training loss decreased much faster. However, the validation loss of the Large CNN reached its minimum around epoch 5 and then began to increase while the training loss continued decreasing. This indicates the beginning of overfitting, because the model continued improving on the training set while its generalization to unseen data became worse.

In contrast, the Small CNN showed slower learning and lower capacity. Although it also exhibited some overfitting later in training, the effect was less extreme because the gap between training and validation performance was smaller. The validation accuracy of the Large CNN remained higher overall (79.97% vs. 74.51%), but the increasing validation loss after epoch 5 demonstrates that the larger architecture was fitting the training data too closely.

Results:

Figure 3.1 comparison of loss between models



The Large CNN overfits because its higher number of layers and parameters gives it enough flexibility to not only learn the real visual patterns in the dataset, but also memorize small details, random variations, and noise that exist only in the training images. Early in training, this helps performance because the model learns meaningful features such as shapes, textures, and edges, so validation accuracy improves. However, once those general patterns are already learned, the extra capacity of the large model allows it to keep reducing training error by fitting information that does not actually help classify new unseen images. This is why after epoch 5 the training loss continues decreasing while the validation loss increases: the model is becoming more specialized to the training set instead of learning features that generalize. The Small CNN has fewer parameters, so it is forced to focus on broader and more general image features and cannot memorize the dataset as aggressively, which reduces overfitting.

In conclusion a larger model initially learns better and reaches higher validation accuracy, but once its capacity exceeds what is necessary for the dataset, the extra parameters begin fitting noise and training specific details rather than useful general patterns. That transition is what causes overfitting.

4. Augmentation

The primary objective of this task is to evaluate the model's spatial invariance and generalization robustness by introducing stochastic geometric transformations into the training pipeline. In standard deep learning workflows, convolutional neural networks can easily overfit to the exact pixel locations and rigid spatial orientations of objects within the dataset. By applying data augmentation, specifically a 50% chance of horizontal flipping and a random rotation of up to 15 degrees (Fig 4.1), we artificially expand the training distribution, forcing the network to learn invariant geometric features rather than memorizing fixed pixel coordinates. To ensure a rigorous and unbiased evaluation of the network's true generalization capacity, these transformations are strictly applied to the training phase, while the validation set is kept entirely clean and unaugmented.

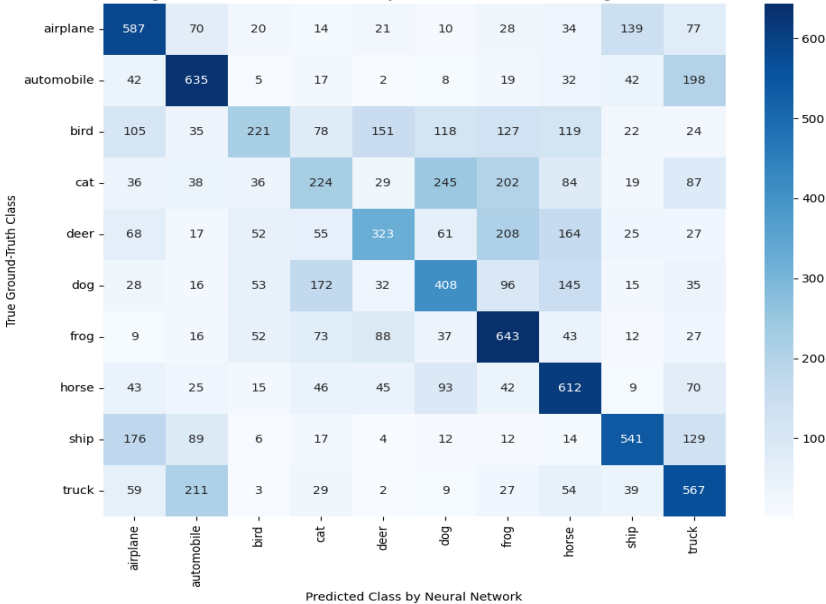
Upon training the Initial CNN architecture for 5 epochs using the augmented data loader, the network demonstrated a stable convergence pattern. The training loss decreased steadily from 1.5764 to 0.4403 while the validation accuracy on the clean test dataset successfully reached 80.06% by the final epoch. Although the introduction of data augmentation successfully mitigates overfitting and stabilizes the generalization gap, the overall learning rate and performance optimization remain inherently bottlenecked by the architectural constraints of the Initial CNN, specifically the vanishing gradient effect caused by the sigmoidal activation functions.

To analyse the specific semantic limitations of the model under geometric variations, a confusion matrix was generated across all 10,000 images of the CIFAR-10 test dataset (Fig. 4.2). The resulting heatmap reveals prominent classification errors between structurally and visually similar biological classes, showing a high rate of confusion between cats and dogs, as well as deers and horses, which indicates that the network struggles to isolate fine-grained discriminative features despite becoming rotation-invariant. Similarly, a significant structural ambiguity is observed within the vehicle categories, where automobiles and trucks are frequently misclassified due to their shared lower-level features such as wheels and horizontal chassis. Conversely, classes with unique silhouettes and distinct background contexts, such as frogs, achieved the highest true positive rates, proving that the augmented model generalizes best on classes with highly isolated pixel distributions.

Figure 4.1: Bounded Data Augmentation Pipeline Execution (CIFAR-10)



Figure 4.2: Confusion Matrix Analysis Under Bounded Data Augmentation



5. Creativity task

The core motivation behind the proposed Patch-Based MLP + CNN Fusion network is to resolve the fundamental trade-offs between Multi-Layer Perceptrons (MLPs) and Convolutional Neural Networks (CNNs) by explicitly separating local feature encoding from global spatial aggregation. Standard MLPs suffer from a massive explosion of parameters when scaling to full images because they lack spatial inductive bias, forcing them to learn pixel interactions based on rigid, absolute coordinates. Conversely, while standard CNNs process localized pixel neighborhoods effectively, they can sometimes be limited in capturing complex, highly non-linear texture combinations in the very earliest stages. To achieve a true synergetic fusion, our architecture fundamentally alters the traditional processing hierarchy by treating the input CIFAR-10 image (32x32x3 pixels) as a collection of localized sub-structures rather than a single flat vector. By slicing the image into a grid of $8 \times 8 = 64$ non-overlapping patches of size $4 \times 4 \times 3$ and passing them through a small, weight-shared MLP subsystem, the network efficiently maps raw pixels into a dense 64-dimensional embedding space without a massive parameter penalty. This stage forces the MLP to act as a highly specialized local texture encoder, learning fine-grained features like micro-gradients and edge orientations within isolated zones, while leaving the task of global geometric synthesis to the subsequent layers. Mathematically, the entire processing pipeline operates deterministically through three main structural stages to handle the data flow. In the first stage, the input tensor is partitioned using an optimized unfolding operation into a matrix of shape [Batch, 64, 48], where 48 represents the flattened pixel values of a single patch, which are then projected through sequential linear layers defined by $\text{Linear}(48 \rightarrow 128) \rightarrow \text{ReLU} \rightarrow \text{Linear}(128 \rightarrow 64) \rightarrow \text{LayerNorm}$. In the second stage, these token embeddings are structurally reconstructed back into a standard four-dimensional tensor format with a spatial grid layout of [Batch, 64, 8, 8], effectively resolving the "border problem" of non-overlapping patches by allowing a deep CNN subsystem to scan the grid using overlapping 3×3 kernels. This convolutional fusion block, implemented as $\text{Conv2D}(64 \rightarrow 128) \rightarrow \text{BatchNorm} + \text{ReLU} \rightarrow \text{MaxPool2d}$ followed by $\text{Conv2D}(128 \rightarrow 256) \rightarrow \text{BatchNorm} + \text{ReLU}$, smoothly blends patch boundaries, downsamples the grid to 4×4 , and establishes broader geometric relationships across distant areas of the image. In the final stage, an AdaptiveAvgPool2d layer compresses the spatial dimensions entirely into a single dense vector of length 256, which represents the global abstract signature of the image and is directly mapped by a fully connected output projection layer ($\text{Linear}(256 \rightarrow 10)$) to generate the 10 class logits. The empirical results collected over a standardized 15-epoch training cycle reveal distinctive behavioural variations, proving the architectural superiority of the hybrid network over both standard baselines (Fig 5.2). As illustrated in the generated training curves (Fig 5.1), the Standard MLP baseline exhibits a clear inability to generalize effectively, plateauing early at a modest test accuracy of approximately 54% due to its lack of spatial translation capacity. Meanwhile, the baseline of the Initial CNN demonstrates a slight learning rate, starting at a low ~24% accuracy and only reaching around 64% by epoch 15, which is caused by the sub-optimal use of Sigmoid activations that severely dampens gradient propagation during backpropagation. In contrast, our proposed Patch-MLP CNN Fusion network optimizes both the initialization quality and overall classification accuracy, establishing a highly stable gradient flow from the very first epoch. While the baseline Initial CNN exhibits a visually steep initial slope, this is merely an artifact of its extremely low starting baseline (25%), and its long-term optimization remains slow and linear due to the gradient-damping effects of legacy Sigmoid activations. Crucially, as depicted in the performance plots, our hybrid architecture bypasses this inefficient early learning phase, instantly establishing a high test accuracy of nearly 60% on epoch 1. Furthermore, while the baseline models quickly stagnate or slow down, the test accuracy curve for our hybrid network shows a consistently steady upward trajectory that maintains a high capacity for generalized feature extraction without early saturation, ultimately achieving an outstanding final test accuracy of over 70%.

Figure 5.1 Training Loss and Test Accuracy comparison between the classic models and the hybrid model.

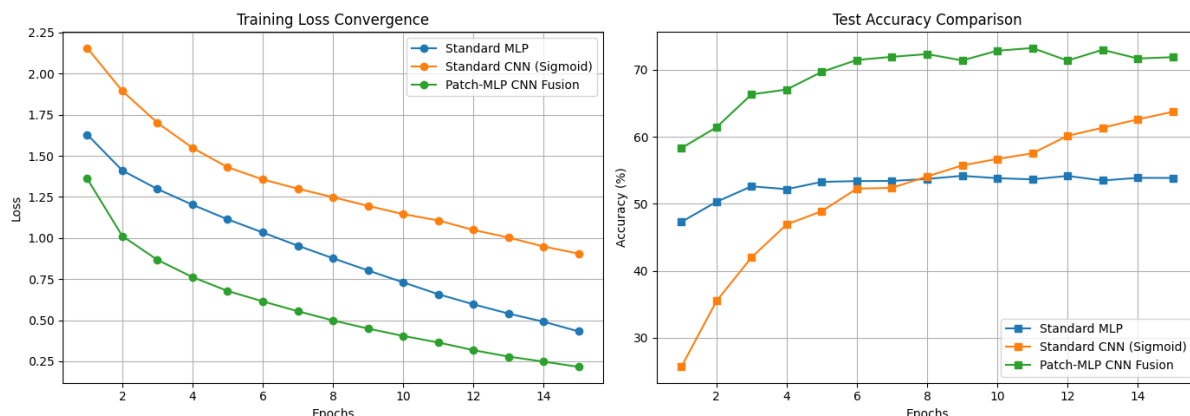
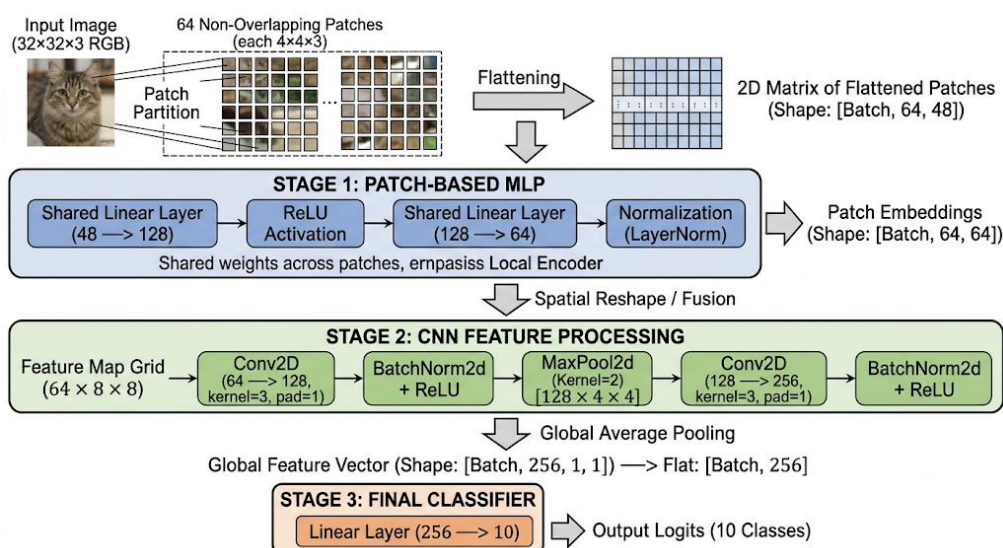


Figure 5.2 Schematic Illustration of the model.



Appendice

Appendix A: Dataset Integrity and Leakage Isolation Audit

To guarantee the statistical validity of our evaluation metrics and ensure that no data leakage occurred during the dataset splitting phase for the 4th question, a rigorous programmatic data integrity audit was conducted on the CIFAR-10 data streams. Every image within both the Training set (50,000 samples) and the Test set (10,000 samples) was converted into a raw byte sequence, and unique MD5 cryptographic hashes were generated for each individual image. A mathematical set intersection operation was then executed over the entire hash space.

As documented in the audit summary, the number of overlapping samples between the training and testing pipelines is 0. This proves that the training data and testing data are entirely disjoint, confirming that the model's final test accuracy represents generalization performance on unseen data without any preprocessing or structural bias.

```
=====
APPENDIX A: DATASET INTEGRITY AUDIT
=====
Total Unique Training Images Registered: 50000
Total Unique Evaluation Testing Images: 10000
Verified Overlapping Intersections Found: 0
Critical Structural Data Leakage Detected: False
=====
Status: SUCCESS - Datasets are completely disjoint.
=====
```